

API 명세서

npm 패키지 비교 의사결정 보조 서비스 Pickage 의 백엔드 REST API 와 내부 RAG 서비스 API

대상	Spring Boot API (backend/) · RAG API (ai/rag/)
엔드포인트	공개 23개 · 운영자 전용 3개 · 내부 1개
기준	커밋 bda8f78 (develop) · 생성 2026-09-24
원본	springdoc OpenAPI(/v3/api-docs), FastAPI openapi() , DTO record 소스에서 자동 생성

1. 공통 규칙

Base URL	https://j15a506.p.ssafy.io/api (로컬 http://localhost:8080/api)
인증	없음 — 공개 조회 서비스. 운영자 API(/api/v1/ops/**)는 nginx 에서 외부 접근을 차단한다
형식	요청·응답 JSON (UTF-8). JSON 필드 이름은 snake_case (Jackson 전역 설정)
배열 파라미터	?names=a&names=b 와 ?names=a,b 를 같게 받는다. 스코프 패키지는 경로가 아니라 쿼리로 보낸다(?names=@types/node)
날짜	YYYY-MM-DD (ISO-8601). 시각은 ISO-8601 UTC
문서 UI	Swagger UI /swagger-ui/index.html · OpenAPI /v3/api-docs

공통 응답 형식

성공	실패
<pre>{ "success": true, "data": { ... 엔드포인트별 응답 ... } }</pre>	<pre>{ "success": false, "code": "V001", "message": "필수 파라미터가 누락되었습니다." }</pre>

이 문서의 **응답 필드** 표는 `data` 안쪽을 기준으로 적는다. 파일을 내려주는 엔드포인트는 래퍼 없이 파일 본문을 그대로 보낸다. 자료가 없음·계산 대상 밖 같은 상태는 오류가 아니라 200 에 `data.status` 값으로 구분한다.

오류 코드

코드	HTTP	message	발생 조건
V001	400	필수 파라미터가 누락되었습니다.	필수 파라미터(names·q·name·refs 등) 누락 또는 빈 값
V002	400	요청 가능한 범위를 넘었습니다.	개수·범위 상한 초과 (names 3개, limit 50, 동시 실행 수 등)
V003	400	날짜 형식이 올바르지 않습니다. (YYYY-MM-DD)	날짜 형식 오류 또는 존재하지 않는 날짜
V004	400	값 형식이 올바르지 않습니다.	값 형식 오류 — 패키지명 규칙·허용되지 않은 enum 값 등
S001	500	일시적인 오류입니다. 잠시 후 다시 시도해 주세요.	서버 내부 오류 (원인은 서버 로그에만 기록)
C005	405	허용되지 않은 http method 접근	HTTP 메서드 불일치 (명세 밖, 프로토콜 수준)
C006	404	요청한 리소스를 찾을 수 없음	없는 경로 또는 만료·미존재 작업 ID
C007	415	지원하지 않는 Content-Type	요청 본문 형식 불일치
C008	406	요청한 Accept에 맞는 응답 형식을 제공할 수 없음	Accept 헤더 불일치

V 로 시작하는 코드는 요청 자체가 규칙을 어긴 것이라 다시 보내도 결과가 같다 — 프런트는 V 코드를 재시도하지 않는다.

2. 엔드포인트 목록

#	분류	Method	URI	기능
1	생태계	GET	/api/packages/search	등록된 패키지명 검색 (포함)
2	생태계	GET	/api/packages/similar	유사 패키지
3	생태계	GET	/api/packages	패키지 개요
4	생태계	GET	/api/packages/downloads	다운로드 추이
5	생태계	GET	/api/packages/dependents	의존 수 추이
6	생태계	GET	/api/packages/version	버전 분포
7	생태계	GET	/api/packages/transitions	유지·유입·이탈
8	생태계	GET	/api/packages/removal-reasons	구간별 이탈 사유
9	생태계	GET	/api/packages/migration-pairs	관측된 교체 흐름 (도착지 분포)
10	생태계	GET	/api/packages/summary	생태계 요약 (실험)
11	기능 비교	GET	/api/packages/versions	기능 비교 버전 목록
12	기능 비교	GET	/api/packages/env	버전별 소비 조건
13	기능 비교	POST	/api/packages/feature-comparison	기능 비교 시작
14	기능 비교	GET	/api/packages/feature-comparison/{runId}	기능 비교 상태
15	커뮤니티	POST	/api/packages/community/refresh	커뮤니티 현황 갱신
16	커뮤니티	GET	/api/packages/community	커뮤니티 현황 조회
17	보고서	POST	/api/report/pdf	PDF 생성
18	보고서	GET	/api/report/pdf/{reportId}	PDF 작업 조회
19	보고서	GET	/api/report/pdf/{reportId}/preview	보고서 미리보기 (HTML)
20	보고서	GET	/api/report/pdf/{reportId}/file	PDF 다운로드
21	보고서	POST	/api/report/markdown	HAND-OFF Markdown 생성
22	보고서	GET	/api/report/markdown/{reportId}	HAND-OFF Markdown 작업 조회
23	보고서	GET	/api/report/markdown/{reportId}/file	HAND-OFF Markdown 다운로드
24	운영자	GET	/api/v1/ops/weekly/runs	최근 회차 목록
25	운영자	GET	/api/v1/ops/weekly/runs/{weekOf}	회차 상세
26	운영자	POST	/api/v1/ops/weekly/runs/{weekOf}/manual-request	수동 실행 요청
27	내부	POST	rag-api:8000/compare	README 근거 기반 공통점·차이 생성

3. 패키지·생태계 변화

3.1 등록된 패키지명 검색 (포함)

GET /api/packages/search

최신 스냅샷이 있는 등록 패키지만 반환한다. 완전 일치·접두사·중간 포함 순, 같은 그룹은 다운로드 순. limit 기본 20, 최대 50.

Request

이름	위치	타입	필수	설명
q	query	string	Y	검색어. 1자 이상 필수(없으면 V001)
limit	query	integer	N	최대 반환 수. 기본 20, 최대 50(초과 V002), 1 미만 V004

Response

200 application/json — data: PackageSearchResponse

필드	타입	설명
data.query	string	서버가 실제로 사용한 검색어. 응답이 늦게 도착했을 때 클라이언트가 현재 입력과 대조해 버릴지 판단하는 데 쓴다.
data.items	array<string>	검색된 패키지명 (완전 일치 → 접두사 → 중간 포함, 같은 그룹은 다운로드 순)

오류 V001 · V002 · V004 · S001

3.2 유사 패키지

GET /api/packages/similar

기준 패키지 하나의 대체 후보를 순위 순으로 반환한다. limit 기본 20, 최대 50. 아직 계산되지 않았으면 200 에 빈 candidates 와 data_status=NO_DATA 로 나간다.

Request

이름	위치	타입	필수	설명
name	query	string	Y	기준 패키지명 1개. 없으면 V001
limit	query	integer	N	최대 반환 수. 기본 20, 최대 50(초과 V002), 1 미만 V004

Response

200 application/json — data: SimilarPackagesResponse

필드	타입	설명
data.base	string	기준 패키지 이름. 요청한 값을 그대로 돌려준다 — 화면이 무엇을 물었는지 알아야 한다.
data.model_ver	string	이 목록을 만든 모델. 스냅샷 날짜 대신 계보를 표시하는 값 이라 화면에 낸다 목록이 비면 null 이다(만든 모델이 없다).
data.data_status	string	COMPLETE 또는 NO_DATA. 후보 retrieval-ranking 실패는 배치가 상태를 남기기 시작하면 여기에 붙는다(구상안 §4.4). 지금은 두 값뿐이다.
data.candidates	array<object>	rank 오름차순. 빈 배열은 아직 계산되지 않았다는 뜻이며 오류가 아니다 기준 이름이 없는 경우와 구분해야 한다 — 그쪽은 not_found 다.
data.candidates[].rank	integer	패키지 안에서 유일하다(UK_SIMILAR_PACKAGE_RANK). 배열 순서와 같지만, 걸러진 후보가 있으면 비어 있는 번호가 생길 수 있으므로 값도 함께 보낸다.

필드	타입	설명
<code>data.candidates[].score</code>	<code>number</code>	유사도에 다른 신호를 더한 종합 점수. 비교용 상대값이지 확률이 아니다 화면에서 "0.9 = 90% 대체 가능" 으로 읽히지 않게 표기해야 한다.
<code>data.candidates[].name</code>	<code>string</code>	패키지명
<code>data.candidates[].latest_version</code>	<code>string</code>	최신 버전. <code>ordinal</code> 로 고른다 — 문자열 정렬하면 <code>4.9.0</code> 이 <code>4.19.2</code> 보다 뒤로 간다.
<code>data.candidates[].description</code>	<code>string</code>	최신 버전의 설명. 원본에 없으면 <code>null</code> 이며 빈 문자열과 구분된다.
<code>data.not_found</code>	<code>array<string></code>	<code>package</code> 테이블에 아예 없는 이름. 있으면 원소 하나뿐이다(기준이 하나이므로).

오류 `V001` · `V002` · `V004` · `S001`

3.3 패키지 개요

`GET` `/api/packages`

이름 배열(최대 3개)로 조회한다. 일부가 없어도 200 이며 `not_found` 에 담긴다. 패키지는 있으나 스냅샷이 없으면 `items` 에 포함되고 지표 필드만 `null` 이다.

Request

이름	위치	타입	필수	설명
<code>names</code>	<code>query</code>	<code>array<string></code>	Y	패키지명 배열. 최대 3개(중복은 하나로 접음). <code>?names=a&names=b</code> 또는 <code>?names=a,b</code> . 없으면 <code>V001</code> , 3개 초과 <code>V002</code> , 형식 위반(214자 초과·허용 문자 밖) <code>V004</code>

Response

200 `application/json` — `data: PackagesOverviewResponse`

필드	타입	설명
<code>data.snapshot_at</code>	<code>string(date)</code>	지표의 기준 스냅샷 날짜 (모든 항목 공통)
<code>data.items</code>	<code>array<object></code>	요청한 이름 순서
<code>data.items[].name</code>	<code>string</code>	패키지명
<code>data.items[].repo_url</code>	<code>string</code>	소스 저장소 주소
<code>data.items[].latest_version</code>	<code>string</code>	최신 정식 버전 (<code>ordinal</code> 기준)
<code>data.items[].published_at</code>	<code>string(date-time)</code>	최신 버전 배포 시각
<code>data.items[].description</code>	<code>string</code>	최신 버전의 <code>package.json</code> description
<code>data.items[].licenses</code>	<code>array<string></code>	라이선스 목록
<code>data.items[].is_deprecated</code>	<code>boolean</code>	최신 버전에 폐기 표시가 있는지. 패키지가 폐기됐다는 뜻이 아니다.
<code>data.items[].downloads</code>	<code>integer</code>	직전 7일 합계. 화면 라벨이 "주간 다운로드" 로 고정돼 있다.
<code>data.items[].stars</code>	<code>integer</code>	GitHub stars (최신 스냅샷)
<code>data.items[].stars_delta</code>	<code>integer</code>	직전 스냅샷 대비 증감. 첫 스냅샷이면 <code>null</code> 이고 화면은 화살표를 숨긴다 52주 누적이 아니다.
<code>data.items[].open_issues</code>	<code>integer</code>	열린 이슈 수 (최신 스냅샷)
<code>data.items[].open_issues_delta</code>	<code>integer</code>	직전 스냅샷 대비 열린 이슈 증감. 첫 스냅샷이면 <code>null</code>
<code>data.not_found</code>	<code>array<string></code>	못 찾은 이름. 일부가 없어도 200 이다.

3.4 다운로드 추이

GET /api/packages/downloads

from 생략 시 최초 스냅샷, to 생략 시 최신 스냅샷. 기간 상한 없음.

Request

이름	위치	타입	필수	설명
names	query	array<string>	Y	패키지명 배열. 최대 3개(중복은 하나로 접음). ?names=a&names=b 또는 ?names=a,b. 없으면 V001, 3개 초과 V002, 형식 위반(214자 초과·허용 문자 밖) V004
from	query	string(date)	N	조회 시작일 YYYY-MM-DD. 생략 시 최초 스냅샷. 형식 오류 V003
to	query	string(date)	N	조회 종료일 YYYY-MM-DD. 생략 시 최신 스냅샷. 형식 오류 V003

Response

200 application/json — data: TrendResponse

필드	타입	설명
data.metric	string	"downloads" 또는 "dependents"
data.unit	string	downloads 전용. 축 라벨의 근거다.
data.sum_over_versions	boolean	dependents 전용. 버전별 합계라 실제 사용처 수보다 크다 기울기는 유효하지만 절대수는 부풀려져 있어, 화면이 축 라벨을 "N개 프로젝트가 사용" 으로 쓰면 안 된다는 신호다.
data.series	array<object>	선 하나. 시리즈마다 길이가 다를 수 있다 신규 패키지는 옛 스냅샷에 행이 아예 없다 없는 점을 0 으로 채워 보내지 않는다 — 채우면 "그 주에 아무도 안 받았다" 가 되어 화면에 없는 급락이 그려진다
data.series[].name	string	패키지명
data.series[].major	string	dependents 전용. 같은 name 이 major 개수만큼 반복된다 downloads 는 버전으로 쪼갤 수 없어 null 이고 직렬화에서 빠진다 존재하지만 구간에 점이 하나도 없는 패키지도 null 이다 — 쪼갤 것이 없기 때문이다
data.series[].points	array<object>	기준일별 값. 없는 기준일은 0 으로 채우지 않고 뺀다 — x축은 snapshot_at 으로 맞춘다
data.series[].points[].snapshot_at	string(date)	스냅샷 기준일
data.series[].points[].value	integer	그 기준일의 값
data.not_found	array<string>	요청했지만 DB 에 없는 패키지명. 일부가 없어도 200 으로 응답한다

3.5 의존 수 추이

GET /api/packages/dependents

스냅샷별로 전 버전의 dependents_count 를 합산한다. 절대수는 부풀려진 값이다.

Request

이름	위치	타입	필수	설명
names	query	array<string>	Y	패키지명 배열. 최대 3개(중복은 하나로 접음). ?names=a&names=b 또는 ?names=a,b. 없으면 V001, 3개 초과 V002, 형식 위반(214자 초과.허용 문자 밖) V004
from	query	string(date)	N	조회 시작일 YYYY-MM-DD. 생략 시 최초 스냅샷. 형식 오류 V003
to	query	string(date)	N	조회 종료일 YYYY-MM-DD. 생략 시 최신 스냅샷. 형식 오류 V003

Response

200 application/json — data: TrendResponse

필드	타입	설명
data.metric	string	"downloads" 또는 "dependents"
data.unit	string	downloads 전용. 축 라벨의 근거다.
data.sum_over_versions	boolean	dependents 전용. 버전별 합계라 실제 사용처 수보다 크다 기울기는 유효하지만 절대수는 부풀려져 있어, 화면이 축 라벨을 "N개 프로젝트가 사용" 으로 쓰면 안 된다는 신호다.
data.series	array<object>	선 하나. 시리즈마다 길이가 다를 수 있다 신규 패키지는 옛 스냅샷에 행이 아예 없다 없는 점을 0 으로 채워 보내지 않는다 — 채우면 "그 주에 아무도 안 받았다" 가 되어 화면에 없는 급락이 그려진다
data.series[].name	string	패키지명
data.series[].major	string	dependents 전용. 같은 name 이 major 개수만큼 반복된다 downloads 는 버전으로 쪼갤 수 없어 null 이고 직렬화에서 빠진다 존재하지만 구간에 점이 하나도 없는 패키지도 null 이다 — 쪼갤 것이 없기 때문이다
data.series[].points	array<object>	기준일별 값. 없는 기준일은 0 으로 채우지 않고 뻗다 — x축은 snapshot_at 으로 맞춘다
data.series[].points[].snapshot_at	string(date)	스냅샷 기준일
data.series[].points[].value	integer	그 기준일의 값
data.not_found	array<string>	요청했지만 DB 에 없는 패키지명. 일부가 없어도 200 으로 응답한다

오류 V001 · V002 · V003 · V004 · S001

3.6 버전 분포

GET /api/packages/version

특정 시점의 major 별 의존 지분. snapshot_at 생략 시 최신 스냅샷. 데이터가 없는 날짜는 200 에 빈 slices 로 나간다.

Request

이름	위치	타입	필수	설명
names	query	array<string>	Y	패키지명 배열. 최대 3개(중복은 하나로 접음). ?names=a&names=b 또는 ?names=a,b. 없으면 V001, 3개 초과 V002, 형식 위반(214자 초과.허용 문자 밖) V004
snapshot_at	query	string(date)	N	기준일 YYYY-MM-DD. 생략 시 최신 스냅샷. 데이터 없는 날짜는 빈 slices(200)

Response

필드	타입	설명
<code>data.snapshot_at</code>	<code>string(date)</code>	스냅샷 기준일
<code>data.basis</code>	<code>string</code>	지분의 기준. 항상 "dependents" 다 — 다운로드는 패키지 단위 단일값이라 버전별로 쪼갤 수 없다.
<code>data.sum_over_versions</code>	<code>boolean</code>	\$5 와 같은 이유로 조각 합계는 부풀려진 값이다 비율은 정상이지만 원 가운데에 총계를 찍으면 안 된다 는 신호.
<code>data.items</code>	<code>array<object></code>	원 하나. <code>slices</code> 가 빈 배열인 것은 에러가 아니다 형식은 맞지만 데이터가 없는 <code>snapshot_at</code> 을 받으면 200 에 빈 배열이 나간다 형식 오류(V003)와 구분해야 한다 — 앞은 "그 날짜엔 자료가 없습니다", 뒤는 "날짜를 다시 쓰세요" 로 화면 안내가 다르다.
<code>data.items[].name</code>	<code>string</code>	패키지명
<code>data.items[].slices</code>	<code>array<object></code>	major 버전별 조각. 데이터가 없는 날짜면 빈 배열
<code>data.items[].slices[].major</code>	<code>string</code>	major 문자열. 0.x 대는 "0" 으로 뭉친다 — 정상 동작이다.
<code>data.items[].slices[].dependents</code>	<code>integer</code>	그 major 를 의존하는 수 (버전별 합계)
<code>data.items[].slices[].pct</code>	<code>number</code>	해당 패키지 안에서의 비율. 패키지별로 각각 100% 가 된다 소수 한 자리라 <code>double</code> 대신 <code>BigDecimal</code> 을 쓴다 — <code>double</code> 은 91.4 를 91.40000000000001 로 직렬화한다.
<code>data.not_found</code>	<code>array<string></code>	요청했지만 DB 에 없는 패키지명. 일부가 없어도 200 으로 응답한다

오류 V001 · V002 · V003 · V004 · S001

3.7 유지·유입·이탈

GET /api/packages/transitions

구간 양 끝의 dependent 선언 집합을 비교한다. period 는 1y·3y·5y 중 하나이며 기본 3y. 한 패키지가 kind(regular·peer·optional)마다 한 줄씩 나온다. 계산 대상 밖이면 0 이 아니라 null 과 data_status=OUT_OF_SCOPE 로 나간다.

Request

이름	위치	타입	필수	설명
names	query	<code>array<string></code>	Y	패키지명 배열. 최대 3개(중복은 하나로 접음). <code>?names=a&names=b</code> 또는 <code>?names=a,b</code> . 없으면 V001, 3개 초과 V002, 형식 위반(214자 초과·허용 문자 밖) V004
period	query	<code>string</code>	N	구간 프리셋 1y · 3y · 5y. 생략 시 3y. 그 밖의 값은 V004

Response

200 application/json — data: TransitionsResponse

필드	타입	설명
<code>data.metric</code>	<code>string</code>	지표 이름
<code>data.period</code>	<code>string</code>	요청한 구간. 생략 시 적용된 기본값을 그대로 돌려준다 — 화면이 무엇을 물었는지 알아야 한다.
<code>data.t1</code>	<code>string(date)</code>	구간 시작 날짜. 그날 23:59:59 까지 발행된 것이 T1 시점이다.
<code>data.t2</code>	<code>string(date)</code>	구간 끝 날짜 = 원천 스냅샷 날짜. "오늘" 이 아니다 — 매일 값이 바뀌면 어제 본 숫자와 달라지므로 고정한다 저장은 23:59:59 까지 담고 여기서 날짜만 잘라 준다 기존 조회들이 <code>snapshot_at</code> 을 날짜로 주는 것과 맞춘다.

필드	타입	설명
<code>data.series</code>	<code>array<object></code>	한 패키지가 <code>kind</code> 수만큼 반복된다 <code>/packages/dependents</code> 가 major 별로 같은 <code>name</code> 을 반복하는 것과 같은 모양이다.
<code>data.series[].name</code>	<code>string</code>	패키지명
<code>data.series[].kind</code>	<code>string</code>	의존 종류 <code>regular</code> · <code>peer</code> · <code>optional</code>
<code>data.series[].population</code>	<code>string</code>	이 수를 센 모집단. 지금은 전부 <code>npm_all</code> (4,065,913).
<code>data.series[].retained</code>	<code>integer</code>	계속 씬 — 구간 안에 새 릴리스를 내면서도 계속 선언했다.
<code>data.series[].inflow</code>	<code>integer</code>	새로 씬 — T1 엔 없고 T2 엔 있다.
<code>data.series[].inflow_new</code>	<code>integer</code>	그중 T1 때 아직 존재하지도 않던 패키지. <code>inflow</code> 의 부분집합.
<code>data.series[].inflow_adopted</code>	<code>integer</code>	<code>inflow - inflowNew</code> . 기존 패키지가 실제로 채택한 수 다 파생값이지만 서버가 직접 주는 이유는, 뽕샘은 자명해도 무엇을 빼야 하는지가 자명하지 않기 때문이다 <code>inflow</code> 를 그대로 그리면 모든 패키지가 다 잘나가는 지표가 되는데, 숫자 자체는 맞아서 틀렸다는 것을 아무도 모른다
<code>data.series[].outflow</code>	<code>integer</code>	그만 씬 — T1 엔 있고 T2 엔 없다.
<code>data.series[].unobserved</code>	<code>integer</code>	판정 불가 — 양 끝에 선언이 있는데 구간 안에 대표 릴리스가 바뀌지 않았다 계속 쓰는지 뺐는지 알 방법이 없다.
<code>data.series[].unobserved_recent</code>	<code>integer</code>	<code>unobserved</code> 중 마지막 대표 릴리스가 <code>t2</code> 기준 3년 안. 셋의 합이 <code>unobserved</code> 다.
<code>data.series[].unobserved_stale</code>	<code>integer</code>	그중 3~5년 전.
<code>data.series[].unobserved_dormant</code>	<code>integer</code>	그중 5년 초과 — 사실상 방치된 프로젝트다 기본 구간(3년) 에서 판정 불가의 63.2% 가 여기다 "판정할 수 없다" 와 "사실상 죽었다" 는 받는 쪽에 전혀 다른 정보다.
<code>data.series[].data_status</code>	<code>string</code>	<code>COMPLETE</code> · <code>NO_DATA</code> · <code>OUT_OF_SCOPE</code> · <code>NOT_COMPUTED</code> . 0 과 "모름" 을 구분하기 위한 값이다.
<code>data.not_found</code>	<code>array<string></code>	<code>package</code> 테이블에 아예 없는 이름. <code>OUT_OF_SCOPE</code> 와 다르다 — 그쪽은 이름은 있는데 계산 대상이 아닌 것이다.

오류 `V001` · `V002` · `V004` · `S001`

3.8 구간별 이탈 사유

`GET` `/api/packages/removal-reasons`

X 를 뺀 전이를 대체 동반(with_replacement)과 대체 없음(no_replacement)으로 가른다. `period` 는 1y·3y·5y 중 하나이며 기본 3y. 한 패키지가 한 줄이다(kind 없음). `unit=transitions` — 패키지 수가 아니라 전이 건수이며 유지·유입·이탈의 수와 더하거나 나누면 안 된다. 대상인데 제거가 없으면 0 과 `NO_DATA`, 대상 밖이면 `null` 과 `OUT_OF_SCOPE` 로 나간다.

Request

이름	위치	타입	필수	설명
<code>names</code>	query	<code>array<string></code>	Y	패키지명 배열. 최대 3개(중복은 하나로 접음). <code>?names=a&names=b</code> 또는 <code>?names=a,b</code> . 없으면 <code>V001</code> , 3개 초과 <code>V002</code> , 형식 위반(214자 초과·허용 문자 밖) <code>V004</code>
<code>period</code>	query	<code>string</code>	N	구간 프리셋 <code>1y</code> · <code>3y</code> · <code>5y</code> . 생략 시 <code>3y</code> . 그 밖의 값은 <code>V004</code>

Response

200 `application/json` — `data: RemovalReasonsResponse`

필드	타입	설명
<code>data.metric</code>	<code>string</code>	지표 이름
<code>data.period</code>	<code>string</code>	요청한 구간. 생략 시 적용된 기본값을 그대로 돌려준다.
<code>data.t1</code>	<code>string(date)</code>	구간 시작 날짜. <code>transitions</code> 응답의 같은 이름과 같은 값이다.
<code>data.t2</code>	<code>string(date)</code>	구간 끝 날짜 = 원천 스냅샷 날짜. "오늘" 이 아니다.
<code>data.series</code>	<code>array<object></code>	한 패키지가 한 줄이다 <code>transitions</code> 와 달리 <code>kind</code> 차원이 없다 — 원천이 선언 종류를 나누지 않는다.
<code>data.series[].name</code>	<code>string</code>	패키지명
<code>data.series[].population</code>	<code>string</code>	이 수를 센 모집단. 지금은 전부 <code>npm_all</code> .
<code>data.series[].unit</code>	<code>string</code>	<code>transitions</code> . 패키지 수가 아니라 전이 건수라는 표시.
<code>data.series[].removals</code>	<code>integer</code>	X 를 뺀 전이의 수.
<code>data.series[].no_replacement</code>	<code>integer</code>	뺀 릴리스에서 아무것도 새로 넣지 않은 전이.
<code>data.series[].with_replacement</code>	<code>integer</code>	뺀 릴리스에서 다른 것을 함께 넣은 전이. 같은 자리의 대체라는 보장은 없다 — 한 릴리스에 섞인 대청소일 수 있다.
<code>data.series[].dependents</code>	<code>integer</code>	X 를 뺀 적 있는 의존자 수 (중복 접음). 위 셋과 단위가 다르다.
<code>data.series[].data_status</code>	<code>string</code>	<code>COMPLETE</code> · <code>NO_DATA</code> · <code>OUT_OF_SCOPE</code> · <code>NOT_COMPUTED</code> .
<code>data.not_found</code>	<code>array<string></code>	<code>package</code> 테이블에 아예 없는 이름.

요류 V001 · V002 · V004 · S001

3.9 관측된 교체 흐름 (도착지 분포)

GET `/api/packages/migration-pairs`

X 를 뺀 릴리스가 무엇을 함께 넣었는지를 도착지 분포로 답한다. `kind` 는 `regular`·`dev` 중 하나이며 기본 `regular` — 모집단이 달라 두 종류의 수를 더하거나 `lift` 절댓값을 비교하면 안 된다. 기본 필터(`votes`>=5 AND `publisher_months`>=3)를 통과한 상위 5개를 세우고 나머지는 `etc` 로 접는다. 점유율 분모는 표가 아니라 조직·달 수(`share_basis=publisher_months`)이며 한 응답의 합이 100 이 아니다. 쌓은 있으나 전부 필터 미달이면 `INSUFFICIENT_EVIDENCE`, 관측이 없으면 `NO_DATA`, `available_package` 에 없으면 `OUT_OF_SCOPE`, 그 종류를 아직 안 올렸으면 `NOT_COMPUTED` 로 나간다.

Request

이름	위치	타입	필수	설명
<code>names</code>	query	<code>array<string></code>	Y	패키지명 배열. 최대 3개(중복은 하나로 접음). <code>?names=a&names=b</code> 또는 <code>?names=a,b</code> . 없으면 V001, 3개 초과 V002, 형식 위반(214자 초과·허용 문자 밖) V004
<code>kind</code>	query	<code>string</code>	N	원천 <code>regular</code> (deps.dev 전수) · <code>dev</code> (registry 상위 10만). 생략 시 <code>regular</code> . 그 밖의 값 V004

Response

200 `application/json` — `data`: `MigrationPairsResponse`

필드	타입	설명
<code>data.metric</code>	<code>string</code>	고정 문자열. 화면이 응답 종류를 분기하는 키다.
<code>data.kind</code>	<code>string</code>	적용된 <code>DependencyKind</code> . 생략 시 기본값을 그대로 돌려준다.
<code>data.series</code>	<code>array<object></code>	한 패키지가 한 줄이다.

필드	타입	설명
<code>data.series[].name</code>	<code>string</code>	패키지명
<code>data.series[].snapshot_at</code>	<code>string(date)</code>	이 종류의 기준일. 읽을 행이 없으면 <code>null</code> .
<code>data.series[].share_basis</code>	<code>string</code>	SHARE_BASIS. 점유율 분모가 표가 아니라는 표시.
<code>data.series[].destinations</code>	<code>array<object></code>	상위 TOP_DESTINATIONS 개. 기본 필터를 통과한 것만, 점유율 내림차순.
<code>data.series[].destinations[].name</code>	<code>string</code>	패키지명
<code>data.series[].destinations[].votes</code>	<code>number</code>	한 전이에 함께 들어온 후보가 k 개면 1/k 씩 나눈 표의 합
<code>data.series[].destinations[].co_events</code>	<code>integer</code>	X 를 빼면서 Y 를 넣은 전이 수
<code>data.series[].destinations[].publisher_months</code>	<code>integer</code>	서로 다른 (배포주체, 달) 조합 수. 한 조직의 일괄 변경을 거른다
<code>data.series[].destinations[].dependents</code>	<code>integer</code>	이 이동을 한 의존자 수 (중복 접음)
<code>data.series[].destinations[].lift</code>	<code>number</code>	모집단 대비 배수. 다른 종류와 절댓값을 비교하지 말 것 — 분모가 그 실행의 전이 수다.
<code>data.series[].destinations[].share_pm_pct</code>	<code>number</code>	점유율. 분모는 SHARE_BASIS 이고, 한 응답 안의 값을 모두 더해도 100 이 되지 않는다 분모가 빌더의 <code>lift>=5</code> 씩 전체인데 표에는 <code>loose(votes>=3)</code> 만 적재되기 때문이다 — 출발 패키지의 79.7% 가 99.5% 에 못 미치고 중앙값은 25.7% 다
<code>data.series[].destinations[].share_pct</code>	<code>number</code>	표 기준 점유율. 라벨 필터용으로만 남긴 값이고 화면에 쓰지 않는다 (S15P21A506-136 §3). 등급 판정의 입력이라 함께 낸다.
<code>data.series[].destinations[].a_pct</code>	<code>number</code>	그 패키지를 뺀 전이 중 이것을 함께 넣은 비율. <code>sharePmPct</code> (도착지들 사이의 몫)와 분모가 다르다 — 이쪽은 이탈 전체가 분모라 "뺀 사람 다섯 중 하나가 이걸 골랐다" 로 읽힌다
<code>data.series[].destinations[].evidence</code>	<code>string</code>	<code>strict</code> · <code>recommended</code> · <code>loose</code> . 행을 지우는 대신 붙이는 배지다(결정 1).
<code>data.series[].destinations[].variant</code>	<code>boolean</code>	양방향 관측. "같은 물건의 두 포장" 일 수 있다 (lodash ↔ lodash-es). 지우지 말고 색만 달리 한다 (결정 4 ③).
<code>data.series[].destinations[].first_seen</code>	<code>string(date)</code>	이 이동이 처음 관측된 날
<code>data.series[].destinations[].last_seen</code>	<code>string(date)</code>	마지막으로 관측된 날
<code>data.series[].etc</code>	<code>object</code>	그 밖 전부를 접은 한 칸. 기본 필터에 못 미친 쌍도 여기 들어간다 — 그래야 <code>destinations + etc</code> 가 이 표가 가진 것 전부가 된다 <code>null</code> 이면 접을 것이 없다
<code>data.series[].etc.pairs</code>	<code>integer</code>	접은 쌍의 수.
<code>data.series[].etc.share_pm_pct</code>	<code>number</code>	접은 쌍의 점유율 합.
<code>data.series[].etc.below_filter</code>	<code>integer</code>	그중 기본 필터에 못 미친 수. 화면이 "근거가 약해 접었다" 를 말할 근거다.
<code>data.series[].observed_pairs</code>	<code>integer</code>	이 패키지에서 관측된 이동쌍 수(필터 전). 0 이면 NO_DATA. 세어 보지 않은 상태에서는 <code>null</code> 이다.

필드	타입	설명
<code>data.series[].data_status</code>	<code>string</code>	<code>COMPLETE</code> · <code>INSUFFICIENT_EVIDENCE</code> · <code>NO_DATA</code> · <code>OUT_OF_SCOPE</code> · <code>NOT_COMPUTED</code> .
<code>data.not_found</code>	<code>array<string></code>	<code>package</code> 표에 아예 없는 이름.

오류 `V001` · `V002` · `V004` · `S001`

3.10 생태계 요약 (실험)

`GET` `/api/packages/summary`

고른 패키지(최대 3개)의 공통 기능과 사용 흐름을 두 문장으로 요약한다. 조합·기준일마다 한 번만 모델을 부르고 이후엔 캐시로 답한다. `status` 가 `READY` 가 아니면 `common-ecosystem` 이 없다.

Request

이름	위치	타입	필수	설명
<code>names</code>	<code>query</code>	<code>array<string></code>	Y	패키지명 배열. 최대 3개(중복은 하나로 접음). <code>?names=a&names=b</code> 또는 <code>?names=a,b</code> . 없으면 <code>V001</code> , 3개 초과 <code>V002</code> , 형식 위반(214자 초과·허용 문자 밖) <code>V004</code>

Response

`200 application/json` — `data: EcosystemSummaryResponse`

필드	타입	설명
<code>data.status</code>	<code>string</code>	<code>READY</code> 면 문장이 있다 <code>UNAVAILABLE</code> (키 미설정) · <code>NO_DATA</code> (찾은 패키지 없음) · <code>FAILED</code> (모델 호출 실패) 면 문장이 <code>null</code> 이고 화면은 안내문을 보여준다.
<code>data.snapshot_at</code>	<code>string(date)</code>	스냅샷 기준일
<code>data.common</code>	<code>string</code>	고른 패키지들이 공통으로 하는 일 1~2문장
<code>data.ecosystem</code>	<code>string</code>	다운로드·의존 등록 수 흐름 1~2문장
<code>data.cached</code>	<code>boolean</code>	이번 응답이 캐시에서 나왔는지 (실험용)
<code>data.usage</code>	<code>object</code>	모델 호출 토큰·크레딧 (실험용, 캐시 응답은 처음 호출 값)
<code>data.usage.input_tokens</code>	<code>integer</code>	입력 토큰 수
<code>data.usage.output_tokens</code>	<code>integer</code>	출력 토큰 수
<code>data.usage.reasoning_tokens</code>	<code>integer</code>	추론 토큰 수
<code>data.usage.credits</code>	<code>number</code>	호출 비용(크레딧)

오류 `V001` · `V002` · `V004` · `S001`

4. 기능 비교

4.1 기능 비교 버전 목록

GET /api/packages/versions

패키지마다 소비 조건이 있는 서로 다른 major의 최신 정식 버전을 최대 3개 최신순으로 돌려준다. 패키지는 있는데 고를 버전이 없으면 versions 가 빈 배열이다.

Request

이름	위치	타입	필수	설명
names	query	array<string>	Y	패키지명 배열. 최대 3개(중복은 하나로 접음). ?names=a&names=b 또는 ?names=a,b. 없으면 V001, 3개 초과 V002, 형식 위반(214자 초과·허용 문자 밖) V004

Response

200 application/json — data: FeatureVersionsResponse

필드	타입	설명
data.packages	array<object>	요청한 순서 그대로. 패키지는 있는데 고를 버전이 없으면 versions 가 빈 배열이고 latestStable 이 null 이다 — 화면은 "비교할 수 있는 버전이 없다" 로 적는다
data.packages[].package_name	string	요청한 이름
data.packages[].latest_stable	string	드롭다운의 기본값. versions 의 맨 앞과 같다 없으면 null
data.packages[].versions	array<string>	최신순. 전부 소비 조건(package_env)이 있는 정식 버전 이라 어느 것을 골라도 핵심 비교 요약이 채워진다
data.not_found	array<string>	package 에 이름 자체가 없는 것

오류 V001 · V002 · V004 · S001

4.2 버전별 소비 조건

GET /api/packages/env

이름@버전 배열(최대 3개). 모듈 방식·타입 동봉·직접/peer 의존 수를 돌려준다. 없는 것은 not_found 에 담기고 200 이다.

Request

이름	위치	타입	필수	설명
refs	query	array<string>	Y	이름@버전 배열. 최대 3개. 없으면 V001, 3개 초과 V002, 형식 위반 V004

Response

200 application/json — data: PackageEnvResponse

필드	타입	설명
data.items	array<object>	요청한 순서 그대로
data.items[].name	string	패키지명
data.items[].version	string	버전

필드	타입	설명
<code>data.items[].module_format</code>	<code>string</code>	어떻게 불러오는가. CJS · ESM_ONLY · ESM_CJS (둘 다) · UNKNOWN (unpublish 라 모름). 전수에서 ESM_CJS 가 21.0% 다 — 화면을 "ESM 이냐 CJS 나" 로 이분해 적으면 다섯 중 하나가 갈 곳이 없다.
<code>data.items[].types_bundled</code>	<code>boolean</code>	타입 선언이 패키지에 동봉됐는가. 거짓은 "타입이 없다" 가 아니라 "이 패키지 안에는 없다" 이다 — <code>@types/xxx</code> 를 따로 깔면 된다 화면 문구는 "별도 설치 필요" 여야 한다.
<code>data.items[].direct_dependencies</code>	<code>integer</code>	<code>dependencies</code> 선언 수. 전이 의존이 아니다 — 31개라고 나와도 실제로 깔리는 것은 수백 개다 화면 라벨에 "직접" 을 반드시 붙인다 <code>null</code> 은 0 이 아니라 모름이다 (unpublish 된 버전은 의존 배열이 통째로 NULL).
<code>data.items[].peer_dependencies</code>	<code>integer</code>	<code>peerDependencies</code> 선언 수. 깔면 따라오는 것이 아니라 사용자가 이미 갖고 있어야 하는 조건 이다 버전이 안 맞으면 설치가 막히거나 경고가 난다 <code>directDependencies</code> 와 더하지 않는다 — 합치면 둘 다 못 읽는다.
<code>data.not_found</code>	<code>array<string></code>	표에 행이 없는 이름@버전. 패키지가 없는 것과 그 버전만 없는 것을 여기서는 구분하지 않는다 — 화면이 할 일이 "빈 칸으로 둔다" 로 같다.

오류 V001 · V002 · V004 · S001

4.3 기능 비교 시작

POST /api/packages/feature-comparison

runId 를 돌려준다. 결과는 상태 조회로 받는다. 동시 실행 수를 넘으면 V002.

Request

이름	위치	타입	필수	설명
refs	query	array<string>	Y	이름@버전 배열. 최대 3개. 없으면 V001, 3개 초과 V002, 형식 위반 V004

Response

200 application/json — data: FeatureRunResponse

필드	타입	설명
<code>data.run_id</code>	<code>string</code>	상태를 물을 때 쓴다
<code>data.status</code>	<code>string</code>	RUNNING · COMPLETED · FAILED
<code>data.phase</code>	<code>string</code>	백엔드가 실제로 지나온 단계. 프론트의 RUN_STEPS 여섯 칸과 대응하지 않는다 — 백엔드가 보는 것은 "RAG 를 불렀다 / 답이 왔다" 뿐이다 여섯 칸을 시간으로 흥내 내면 끝나지 않은 단계를 완료로 그린다.
<code>data.refs</code>	array<string>	비교 대상. 요청 순서 그대로
<code>data.elapsed_sec</code>	integer	시작 후 흐른 초. 화면이 대기 표시를 그릴 때 쓴다
<code>data.result</code>	object(JSON 원문)	COMPLETED 일 때만. RAG 응답을 그대로 싣는다 — 그 계약의 주인은 ai/rag/main.py 이고, 여기서 한 벌 더 정의하면 한쪽만 고쳐지는 날이 온다 키 이름도 camelCase 그대로 나간다
<code>data.error_code</code>	string	FAILED 일 때만. DOC_NOT_FOUND (그 버전의 문헌이 아직 없다) · VERIFICATION_FAILED (생성물이 근거와 어긋나 막았다) · RAG_UNAVAILABLE result · errorDetail 은 JSON 원문을 그대로 싣는다(JsonRawValue).
<code>data.error_detail</code>	object(JSON 원문)	FAILED 일 때 RAG 가 준 상세(JSON 원문). 예: 검증 위반 목록

`data.result` 는 RAG API `/compare` 응답 원문을 그대로 싣는다(키는 camelCase). 구조는 RAG 비교 응답을 따른다.

오류 `V001` · `V002` (인자 초과 또는 동시 실행 수 초과) · `V004` · `S001`

4.4 기능 비교 상태

GET `/api/packages/feature-comparison/{runId}`

COMPLETED 면 `result` 가 찬다. 실패도 200 이며 `status=FAILED` · `error_code` 로 구분한다.

Request

이름	위치	타입	필수	설명
<code>runId</code>	<code>path</code>	<code>string</code>	Y	기능 비교 시작 응답의 <code>run_id</code> . 없거나 만료되면 404(C006)

Response

200 `application/json` — `data: FeatureRunResponse`

필드	타입	설명
<code>data.run_id</code>	<code>string</code>	상태를 물을 때 쓴다
<code>data.status</code>	<code>string</code>	<code>RUNNING</code> · <code>COMPLETED</code> · <code>FAILED</code>
<code>data.phase</code>	<code>string</code>	백엔드가 실제로 지나온 단계. 프런트의 <code>RUN_STEPS</code> 여섯 칸과 대응하지 않는다 — 백엔드가 보는 것은 "RAG 를 불렀다 / 답이 왔다" 뿐이다 여섯 칸을 시간으로 흥내 내면 끝나지 않은 단계를 완료로 그린다.
<code>data.refs</code>	<code>array<string></code>	비교 대상. 요청 순서 그대로
<code>data.elapsed_sec</code>	<code>integer</code>	시작 후 흐른 초. 화면이 대기 표시를 그릴 때 쓴다
<code>data.result</code>	<code>object(JSON 원문)</code>	COMPLETED 일 때만. RAG 응답을 그대로 싣는다 — 그 계약의 주인은 <code>ai/rag/main.py</code> 이고, 여기서 한 벌 더 정의하면 한쪽만 고쳐지는 날이 온다 키 이름도 camelCase 그대로 나간다
<code>data.error_code</code>	<code>string</code>	FAILED 일 때만. <code>DOC_NOT_FOUND</code> (그 버전의 문헌이 아직 없다) · <code>VERIFICATION_FAILED</code> (생성물이 근거와 어긋나 막았다) · <code>RAG_UNAVAILABLE</code> <code>result</code> · <code>errorDetail</code> 은 JSON 원문을 그대로 싣는다(JsonRawValue).
<code>data.error_detail</code>	<code>object(JSON 원문)</code>	FAILED 일 때 RAG 가 준 상세(JSON 원문). 예: 검증 위반 목록

`data.result` 는 RAG API `/compare` 응답 원문을 그대로 싣는다(키는 camelCase). 구조는 RAG 비교 응답을 따른다.

오류 `C006` · `S001`

5. GitHub 커뮤니티

5.1 커뮤니티 현황 갱신

POST /api/packages/community/refresh

새 작업을 수락하면 202, 이미 신선한 결과가 있거나 같은 작업에 참여하거나 용량이 초과되면 200이다.

Request

이름	위치	타입	필수	설명
name	query	string	Y	기준 패키지명 1개. 없으면 V001
trigger	query	string(enum)	Y	갱신 계기. ANALYSIS_CONFIRMED (비교 대상 확정 직후) · TAB_OPENED (탭 진입)

Response

202 새 작업 수락 · 200 신선한 결과 있음 · 기존 작업 참여 · 용량 초과 application/json — data:

CommunityStatusResponse

필드	타입	설명
data.package_name	string	패키지명
data.view_status	string(enum)	값: IDLE · PROCESSING · RESULT · FAILED
data.freshness	string(enum)	값: FRESH · STALE
data.refresh	object	진행 중이거나 마지막 갱신 작업 정보. 작업이 없으면 null
data.refresh.refresh_id	string(uuid)	갱신 작업 식별자
data.refresh.status	string(enum)	값: QUEUED · RUNNING · COMPLETED · FAILED · CAPACITY_LIMITED
data.refresh.stage	string	진행 단계 (예: COLLECTING_DISCUSSIONS)
data.refresh.stage_message	string	화면에 보여줄 단계 안내 문구
data.refresh.started_at	string(date-time)	시작 시각
data.refresh.last_updated_at	string(date-time)	단계가 마지막으로 바뀐 시각
data.refresh.poll_after_seconds	integer	다음 조회까지 기다릴 초 (폴링 간격)
data.refresh.retry_at	string(date-time)	실패 · 용량 초과 시 다시 시도할 수 있는 시각
data.refresh.error_code	string(enum)	값: GITHUB_RATE_LIMITED · GITHUB_UNAVAILABLE · NPM_UNAVAILABLE · GMS_UNAVAILABLE · REFRESH_DEADLINE_EXCEEDED · PUBLISH_FAILED · CAPACITY_LIMITED · LOCAL_RATE_LIMITED · COMMUNITY_DISABLED
data.result	object	저장된 분석 결과. 아직 없으면 null
data.result.snapshot_id	string(uuid)	결과 스냅샷 식별자. 동시 갱신 결과를 구분한다
data.result.collected_at	string(date-time)	수집 시각. 재사용 · 제공 기간 판정의 기준
data.result.fresh_until	string(date-time)	이 시각까지는 새로 수집하지 않고 재사용한다 (수집 후 24시간)
data.result.serve_until	string(date-time)	이 시각까지 결과를 제공한다 (수집 후 7일)

필드	타입	설명
<code>data.result.data_status</code>	<code>string(enum)</code>	값: AVAILABLE · PARTIAL · UNVERIFIED_REPOSITORY · AMBIGUOUS_SCOPE · UNSUPPORTED_HOST · NO_DISCUSSION_DATA
<code>data.result.summary_status</code>	<code>string(enum)</code>	값: READY · PARTIAL · FAILED · SKIPPED
<code>data.result.summary_retry_at</code>	<code>string(date-time)</code>	요약 생성 실패 시 재시도 가능 시각
<code>data.result.repository</code>	<code>object</code>	검증된 GitHub 저장소 정보
<code>data.result.repository.owner</code>	<code>string</code>	저장소 소유자
<code>data.result.repository.name</code>	<code>string</code>	저장소 이름
<code>data.result.repository.full_name</code>	<code>string</code>	owner/name
<code>data.result.repository.scope</code>	<code>string</code>	패키지와 저장소의 대응 범위 (단일 패키지 / 모노레포 등)
<code>data.result.repository.archived</code>	<code>boolean</code>	저장소 보관(archived) 여부
<code>data.result.repository.issue_count</code>	<code>integer</code>	저장소 전체 Issue 수 (PR 제외). 못 구했으면 null
<code>data.result.repository.open_issue_count</code>	<code>integer</code>	그중 열린 Issue 수
<code>data.result.summary</code>	<code>object</code>	수집 범위 안의 활동 합계
<code>data.result.summary.issue_count</code>	<code>integer</code>	수집한 Issue 수
<code>data.result.summary.comment_count</code>	<code>integer</code>	수집한 댓글 수
<code>data.result.summary.reaction_count</code>	<code>integer</code>	수집한 반응(이모지) 수
<code>data.result.summary.open_issue_count</code>	<code>integer</code>	수집한 Issue 중 열린 수
<code>data.result.topics</code>	<code>array<object></code>	핵심 논의 Issue (최대 2개)
<code>data.result.topics[].issue_number</code>	<code>integer</code>	GitHub Issue 번호
<code>data.result.topics[].state</code>	<code>string</code>	Issue 상태 open · closed
<code>data.result.topics[].updated_at</code>	<code>string(date-time)</code>	마지막 갱신 시각
<code>data.result.topics[].created_at</code>	<code>string(date-time)</code>	생성 시각
<code>data.result.topics[].title_original</code>	<code>string</code>	Issue 원문 제목
<code>data.result.topics[].title_ko</code>	<code>string</code>	한국어 제목
<code>data.result.topics[].comments_count</code>	<code>integer</code>	댓글 수
<code>data.result.topics[].reactions_count</code>	<code>integer</code>	반응 수
<code>data.result.topics[].collection_status</code>	<code>string(enum)</code>	값: COMPLETE · TRUNCATED · FAILED
<code>data.result.topics[].summary_status</code>	<code>string(enum)</code>	값: READY · PARTIAL · FAILED · SKIPPED
<code>data.result.topics[].summary_ko</code>	<code>string</code>	논의 한국어 요약 (LLM 생성)
<code>data.result.topics[].messages</code>	<code>array<object></code>	대표 발화
<code>data.result.topics[].messages[].author_login</code>	<code>string</code>	작성자 GitHub 로그인
<code>data.result.topics[].messages[].role</code>	<code>string</code>	작성자 역할 (메인테이너·기여자·사용자 등)
<code>data.result.topics[].messages[].kind</code>	<code>string</code>	발화 종류 (본문·댓글)
<code>data.result.topics[].messages[].created_at</code>	<code>string(date-time)</code>	생성 시각
<code>data.result.topics[].messages[].text</code>	<code>string</code>	발화 원문 발체

필드	타입	설명
<code>data.result.topics[].summary_marks</code>	<code>array<object></code>	요약문 강조 구간
<code>data.result.topics[].summary_marks[].start</code>	<code>integer</code>	시작 오프셋 (UTF-16, 포함)
<code>data.result.topics[].summary_marks[].end</code>	<code>integer</code>	끝 오프셋 (UTF-16, 미포함)
<code>data.result.topics[].summary_marks[].kind</code>	<code>string</code>	강조 종류 (핵심 문장·핵심어)
<code>data.result.limitations</code>	<code>array<object></code>	결과 제한 사유 목록
<code>data.result.limitations[].code</code>	<code>string</code>	제한 사유 코드
<code>data.result.limitations[].message</code>	<code>string</code>	제한 사유 안내 문구
<code>data.result.limitations[].issue_number</code>	<code>integer</code>	특정 Issue 에 대한 제한이면 그 번호
<code>data.result.data_limits</code>	<code>object</code>	수집 정책 한도
<code>data.result.data_limits.policy_version</code>	<code>string</code>	수집 정책 버전
<code>data.result.data_limits.lookback_days</code>	<code>integer</code>	수집 대상 기간(일)
<code>data.result.data_limits.max_issues</code>	<code>integer</code>	수집하는 최대 Issue 수
<code>data.result.data_limits.max_comments_per_issue</code>	<code>integer</code>	Issue 당 최대 댓글 수
<code>data.result.data_limits.max_messages_per_issue</code>	<code>integer</code>	Issue 당 대표 발화 최대 수
<code>data.result.data_limits.source_note</code>	<code>string</code>	출처 안내 문구

오류 V001 · V004 · S001

5.2 커뮤니티 현황 조회

GET `/api/packages/community`

진행 상태 또는 저장된 결과만 읽는다. 새 수집을 시작하지 않는다.

Request

이름	위치	타입	필수	설명
<code>name</code>	<code>query</code>	<code>string</code>	Y	기준 패키지명 1개. 없으면 V001

Response

200 `application/json` — `data: CommunityStatusResponse`

필드	타입	설명
<code>data.package_name</code>	<code>string</code>	패키지명
<code>data.view_status</code>	<code>string(enum)</code>	값: <code>IDLE</code> · <code>PROCESSING</code> · <code>RESULT</code> · <code>FAILED</code>
<code>data.freshness</code>	<code>string(enum)</code>	값: <code>FRESH</code> · <code>STALE</code>
<code>data.refresh</code>	<code>object</code>	진행 중이거나 마지막 갱신 작업 정보. 작업이 없으면 null
<code>data.refresh.refresh_id</code>	<code>string(uuid)</code>	갱신 작업 식별자
<code>data.refresh.status</code>	<code>string(enum)</code>	값: <code>QUEUED</code> · <code>RUNNING</code> · <code>COMPLETED</code> · <code>FAILED</code> · <code>CAPACITY_LIMITED</code>
<code>data.refresh.stage</code>	<code>string</code>	진행 단계 (예: <code>COLLECTING_DISCUSSIONS</code>)
<code>data.refresh.stage_message</code>	<code>string</code>	화면에 보여줄 단계 안내 문구
<code>data.refresh.started_at</code>	<code>string(date-time)</code>	시작 시각

필드	타입	설명
<code>data.refresh.last_updated_at</code>	<code>string(date-time)</code>	단계가 마지막으로 바뀐 시각
<code>data.refresh.poll_after_seconds</code>	<code>integer</code>	다음 조회까지 기다릴 초 (폴링 간격)
<code>data.refresh.retry_at</code>	<code>string(date-time)</code>	실패·용량 초과 시 다시 시도할 수 있는 시각
<code>data.refresh.error_code</code>	<code>string(enum)</code>	값: <code>GITHUB_RATE_LIMITED</code> · <code>GITHUB_UNAVAILABLE</code> · <code>NPM_UNAVAILABLE</code> · <code>GMS_UNAVAILABLE</code> · <code>REFRESH_DEADLINE_EXCEEDED</code> · <code>PUBLISH_FAILED</code> · <code>CAPACITY_LIMITED</code> · <code>LOCAL_RATE_LIMITED</code> · <code>COMMUNITY_DISABLED</code>
<code>data.result</code>	<code>object</code>	저장된 분석 결과. 아직 없으면 null
<code>data.result.snapshot_id</code>	<code>string(uuid)</code>	결과 스냅샷 식별자. 동시 갱신 결과를 구분한다
<code>data.result.collected_at</code>	<code>string(date-time)</code>	수집 시각. 재사용·제공 기간 판정의 기준
<code>data.result.fresh_until</code>	<code>string(date-time)</code>	이 시각까지는 새로 수집하지 않고 재사용한다 (수집 후 24시간)
<code>data.result.serve_until</code>	<code>string(date-time)</code>	이 시각까지 결과를 제공한다 (수집 후 7일)
<code>data.result.data_status</code>	<code>string(enum)</code>	값: <code>AVAILABLE</code> · <code>PARTIAL</code> · <code>UNVERIFIED_REPOSITORY</code> · <code>AMBIGUOUS_SCOPE</code> · <code>UNSUPPORTED_HOST</code> · <code>NO_DISCUSSION_DATA</code>
<code>data.result.summary_status</code>	<code>string(enum)</code>	값: <code>READY</code> · <code>PARTIAL</code> · <code>FAILED</code> · <code>SKIPPED</code>
<code>data.result.summary_retry_at</code>	<code>string(date-time)</code>	요약 생성 실패 시 재시도 가능 시각
<code>data.result.repository</code>	<code>object</code>	검증된 GitHub 저장소 정보
<code>data.result.repository.owner</code>	<code>string</code>	저장소 소유자
<code>data.result.repository.name</code>	<code>string</code>	저장소 이름
<code>data.result.repository.full_name</code>	<code>string</code>	owner/name
<code>data.result.repository.scope</code>	<code>string</code>	패키지와 저장소의 대응 범위 (단일 패키지 / 모노레포 등)
<code>data.result.repository.archived</code>	<code>boolean</code>	저장소 보관(archived) 여부
<code>data.result.repository.issue_count</code>	<code>integer</code>	저장소 전체 Issue 수 (PR 제외). 못 구했으면 null
<code>data.result.repository.open_issue_count</code>	<code>integer</code>	그중 열린 Issue 수
<code>data.result.summary</code>	<code>object</code>	수집 범위 안의 활동 합계
<code>data.result.summary.issue_count</code>	<code>integer</code>	수집한 Issue 수
<code>data.result.summary.comment_count</code>	<code>integer</code>	수집한 댓글 수
<code>data.result.summary.reaction_count</code>	<code>integer</code>	수집한 반응(이모지) 수
<code>data.result.summary.open_issue_count</code>	<code>integer</code>	수집한 Issue 중 열린 수
<code>data.result.topics</code>	<code>array<object></code>	핵심 논의 Issue (최대 2개)
<code>data.result.topics[].issue_number</code>	<code>integer</code>	GitHub Issue 번호
<code>data.result.topics[].state</code>	<code>string</code>	Issue 상태 <code>open</code> · <code>closed</code>
<code>data.result.topics[].updated_at</code>	<code>string(date-time)</code>	마지막 갱신 시각
<code>data.result.topics[].created_at</code>	<code>string(date-time)</code>	생성 시각

필드	타입	설명
<code>data.result.topics[].title_original</code>	string	Issue 원문 제목
<code>data.result.topics[].title_ko</code>	string	한국어 제목
<code>data.result.topics[].comments_count</code>	integer	댓글 수
<code>data.result.topics[].reactions_count</code>	integer	반응 수
<code>data.result.topics[].collection_status</code>	string(enum)	값: COMPLETE · TRUNCATED · FAILED
<code>data.result.topics[].summary_status</code>	string(enum)	값: READY · PARTIAL · FAILED · SKIPPED
<code>data.result.topics[].summary_ko</code>	string	논의 한국어 요약 (LLM 생성)
<code>data.result.topics[].messages</code>	array<object>	대표 발화
<code>data.result.topics[].messages[].author_login</code>	string	작성자 GitHub 로그인
<code>data.result.topics[].messages[].role</code>	string	작성자 역할 (메인테이너·기여자·사용자 등)
<code>data.result.topics[].messages[].kind</code>	string	발화 종류 (본문·댓글)
<code>data.result.topics[].messages[].created_at</code>	string(date-time)	생성 시각
<code>data.result.topics[].messages[].text</code>	string	발화 원문 발체
<code>data.result.topics[].summary_marks</code>	array<object>	요약문 강조 구간
<code>data.result.topics[].summary_marks[].start</code>	integer	시작 오프셋 (UTF-16, 포함)
<code>data.result.topics[].summary_marks[].end</code>	integer	끝 오프셋 (UTF-16, 미포함)
<code>data.result.topics[].summary_marks[].kind</code>	string	강조 종류 (핵심 문장·핵심어)
<code>data.result.limitations</code>	array<object>	결과 제한 사유 목록
<code>data.result.limitations[].code</code>	string	제한 사유 코드
<code>data.result.limitations[].message</code>	string	제한 사유 안내 문구
<code>data.result.limitations[].issue_number</code>	integer	특정 Issue 에 대한 제한이면 그 번호
<code>data.result.data_limits</code>	object	수집 정책 한도
<code>data.result.data_limits.policy_version</code>	string	수집 정책 버전
<code>data.result.data_limits.lookback_days</code>	integer	수집 대상 기간(일)
<code>data.result.data_limits.max_issues</code>	integer	수집하는 최대 Issue 수
<code>data.result.data_limits.max_comments_per_issue</code>	integer	Issue 당 최대 댓글 수
<code>data.result.data_limits.max_messages_per_issue</code>	integer	Issue 당 대표 발화 최대 수
<code>data.result.data_limits.source_note</code>	string	출처 안내 문구

오류 V001 · V004 · S001

6. 보고서 출력 (PDF · HAND-OFF)

6.1 PDF 생성

POST /api/report/pdf

비교 대상과 조회 조건을 받아 문서를 만든다. 숫자는 서버가 다시 조회하므로 화면이 들고 있던 옛 값이 문서에 박히지 않는다. 응답은 작업 정보이며 파일이 아니다.

Request

Body application/json — PdfGenerateRequest

필드	타입	설명
names	array<string>	비교 대상. 기준 패키지를 포함해 최대 3개.
from	string(date)	생태계 조회 시작. 생략하면 서버 기본 구간.
to	string(date)	생태계 조회 끝.
snapshot_at	string(date)	버전 분포 기준일. 생략하면 최신 스냅샷.
period	string	유지·유입·이탈 조회 구간(1y·3y·5y). 생략하면 서버 기본값 (TransitionPeriod.DEFAULT, 3y). from·to 와는 다른 축이다 — 그쪽은 시계열 구간이고 이쪽은 두 시점을 비교한 프리셋이라 같은 날짜로 겹쳐 쓸 수 없다(S15P21A506-391).
sections	array<string>	더할 구역 (COMMUNITY · FEATURES). 생략하면 생태계만. 생태계는 여기 넣지 않는다 — 끝 수 없는 것을 고를 수 있게 두면 "생태계 빼고 만들기" 라는 없는 상태가 생긴다.
features	object	sections 에 FEATURES 를 넣었을 때, 세션이 들고 있는 완료 기능 비교 결과(구상안 §13.1-§14.5). 서버는 이 값을 재조회하지 않고 그대로 문서에 옮긴다(DEC-FEATURE-CACHE-20260917-01) — 판정을 서버에 영속화하지 않기 때문이다
features.packages	array<object>	비교 대상 패키지·버전
features.packages[].package_name	string	패키지명
features.packages[].version	string	버전
features.common	string	공통점 서술
features.differences	array<object>	패키지별 차이 문단
features.differences[].package_name	string	패키지명
features.differences[].version	string	버전
features.differences[].body	string	차이 본문
features.limited	boolean	자료 제한 여부 (README 가 짧거나 없어 비교가 제한됨)

Response

200 application/json — data: PdfJobResponse

필드	타입	설명
data.report_id	string	다운로드에 쓰는 식별자.
data.status	string	지금은 항상 COMPLETE .
data.file_name	string	사용자에게 보일 이름(구상안 §13.6). 저장된 파일 이름과 다르다.
data.bytes	integer	파일 크기. 화면이 "크기 표시"(IA §12.3)에 쓴다.

필드	타입	설명
<code>data.created_at</code>	<code>string(date-time)</code>	생성 시각.
<code>data.omitted</code>	<code>array<string></code>	요청했지만 문서에 채우지 못한 구역. 기능이 아직 없어서 비운 것이며 오류가 아니다 조용히 빼면 사용자는 체크한 것이 사라진 이유를 알 수 없다 — <code>not_found</code> 를 따로 신는 것과 같은 이유다

오류 V001 · V004 (알 수 없는 sections 값) · S001

6.2 PDF 작업 조회

GET `/api/report/pdf/{reportId}`

생성된 문서의 파일명·크기·생성 시각. 다시 받기 전에 상태를 확인하는 자리다.

Request

이름	위치	타입	필수	설명
<code>reportId</code>	path	<code>string</code>	Y	생성 응답의 <code>report_id</code> . 없거나 만료되면 404(C006)

Response

200 `application/json` — `data: PdfJobResponse`

필드	타입	설명
<code>data.report_id</code>	<code>string</code>	다운로드에 쓰는 식별자.
<code>data.status</code>	<code>string</code>	지금은 항상 <code>COMPLETE</code> .
<code>data.file_name</code>	<code>string</code>	사용자에게 보일 이름(구상안 §13.6). 저장된 파일 이름과 다르다.
<code>data.bytes</code>	<code>integer</code>	파일 크기. 화면이 "크기 표시"(IA §12.3)에 쓴다.
<code>data.created_at</code>	<code>string(date-time)</code>	생성 시각.
<code>data.omitted</code>	<code>array<string></code>	요청했지만 문서에 채우지 못한 구역. 기능이 아직 없어서 비운 것이며 오류가 아니다 조용히 빼면 사용자는 체크한 것이 사라진 이유를 알 수 없다 — <code>not_found</code> 를 따로 신는 것과 같은 이유다

오류 C006 · S001

6.3 보고서 미리보기 (HTML)

GET `/api/report/pdf/{reportId}/preview`

문서 내용을 HTML 로 반환한다. 다운로드할 PDF 와 같은 생성에서 나온 것이다.

Request

이름	위치	타입	필수	설명
<code>reportId</code>	path	<code>string</code>	Y	생성 응답의 <code>report_id</code> . 없거나 만료되면 404(C006)

Response

200 `text/html; charset=UTF-8` — 보고서 미리보기 HTML 문서

오류 C006 · S001

6.4 PDF 다운로드

GET `/api/report/pdf/{reportId}/file`

같은 `reportId` 로 여러 번 받을 수 있고, 다시 받아도 보고서를 새로 만들지 않는다.

Request

이름	위치	타입	필수	설명
reportId	path	string	Y	생성 응답의 report_id. 없거나 만료되면 404(C006)

Response

200 application/pdf — 파일 본문. Content-Disposition: attachment; filename*=UTF-8'...'

오류 C006 · S001

6.5 HAND-OFF Markdown 생성

POST /api/report/markdown

비교 대상과 조회 조건을 받아 agent 친화적 .md 문서를 만든다. 숫자는 서버가 다시 조회하므로 화면이 들고 있던 옛 값이 문서에 박히지 않는다. 구역은 항상 전체를 시도한다(선택 UI 없음). 응답은 작업 정보이며 파일이 아니다.

Request

Body application/json — MarkdownGenerateRequest

필드	타입	설명
names	array<string>	비교 대상. 기준 패키지를 포함해 최대 3개.
from	string(date)	생태계 조회 시작. 생략하면 서버 기본 구간.
to	string(date)	생태계 조회 끝.
snapshot_at	string(date)	버전 분포 기준일. 생략하면 최신 스냅샷.
period	string	유지·유입·이탈 조회 구간. 생략하면 서버 기본값(3y).
features	object	세션이 들고 있는 완료 기능 비교 결과. 서버는 재조회하지 않고 그대로 옮긴다(DEC-FEATURE-CACHE-20260917-01). 없으면(아직 분석 전) 그 사실을 문서와 응답(omitted)에 적는다 — PdfGenerateRequest 와 같은 계약.
features.packages	array<object>	비교 대상 패키지·버전
features.packages[].package_name	string	패키지명
features.packages[].version	string	버전
features.common	string	공통점 서술
features.differences	array<object>	패키지별 차이 문단
features.differences[].package_name	string	패키지명
features.differences[].version	string	버전
features.differences[].body	string	차이 본문
features.limited	boolean	자료 제한 여부 (README 가 짧거나 없어 비교가 제한됨)

Response

200 application/json — data: MarkdownJobResponse

필드	타입	설명
data.report_id	string	다운로드에 쓰는 식별자.
data.file_name	string	사용자에게 보일 이름(.md).
data.bytes	integer	파일 크기.
data.created_at	string(date-time)	생성 시각.

필드	타입	설명
<code>data.omitted</code>	<code>array<string></code>	요청했지만 문서에 채우지 못한 구역. PdfJobResponseomitted 와 같은 뜻.

오류 V001 · S001

6.6 HAND-OFF Markdown 작업 조회

GET `/api/report/markdown/{reportId}`

생성된 문서의 파일명·크기·생성 시각.

Request

이름	위치	타입	필수	설명
<code>reportId</code>	path	<code>string</code>	Y	생성 응답의 report_id. 없거나 만료되면 404(C006)

Response

200 application/json — data: MarkdownJobResponse

필드	타입	설명
<code>data.report_id</code>	<code>string</code>	다운로드에 쓰는 식별자.
<code>data.file_name</code>	<code>string</code>	사용자에게 보일 이름(.md).
<code>data.bytes</code>	<code>integer</code>	파일 크기.
<code>data.created_at</code>	<code>string(date-time)</code>	생성 시각.
<code>data.omitted</code>	<code>array<string></code>	요청했지만 문서에 채우지 못한 구역. PdfJobResponseomitted 와 같은 뜻.

오류 C006 · S001

6.7 HAND-OFF Markdown 다운로드

GET `/api/report/markdown/{reportId}/file`

같은 reportId 로 여러 번 받을 수 있고, 다시 받아도 문서를 새로 만들지 않는다.

Request

이름	위치	타입	필수	설명
<code>reportId</code>	path	<code>string</code>	Y	생성 응답의 report_id. 없거나 만료되면 404(C006)

Response

200 text/markdown; charset=UTF-8 — 파일 본문. Content-Disposition: attachment; filename*=UTF-8'...'

오류 C006 · S001

7. 운영자 전용 — 주간 수집

운영자 전용. nginx 가 /api/v1/ops/ 를 외부에 막고 운영자는 SSH 터널로 접근한다. Swagger 문서에서도 숨겨져 있어 (@Hidden) 이 명세는 컨트롤러 소스에서 만들었다.

7.1 최근 회차 목록

GET /api/v1/ops/weekly/runs

최신 주부터 거슬러 올라간다. 타이머가 한 번도 돌지 않아 상태 객체가 없는 주도 status=MISSING 으로 포함한다 — 저장된 것만 보여 주면 그 주가 화면에서 사라지는데, 그게 가장 알아야 할 주다.

Request

이름	위치	타입	필수	설명
weeks	query	integer	N	최신 주부터 거슬러 볼 주 수. 생략 시 기본값

Response

200 application/json — data: List<WeeklyRunResponse>

필드	타입	설명
data[].week_of	string(date)	회차 주의 월요일 (deps.dev 스냅샷 날짜)
data[].status	string	PENDING · RUNNING · SUCCEEDED · FAILED · BLOCKED, 그리고 MISSING — 그 주에 상태 객체가 아예 없다는 뜻이다 타이머가 한 번도 돌지 않았거나 멈춰 있었다
data[].coverage	object	이 회차가 덮는 데이터 범위
data[].coverage.depsdev_snapshot	string(date)	deps.dev 스냅샷 날짜
data[].coverage.downloads_through	string(date)	다운로드 수집 마지막 날짜
data[].coverage.window_start	string(date)	수집 구간 시작
data[].coverage.window_end	string(date)	수집 구간 끝
data[].started_at	string(date-time)	시작 시각
data[].finished_at	string(date-time)	종료 시각
data[].consecutive_failures	integer	연속 실패 횟수. 한도를 넘으면 BLOCKED
data[].last_error	string	마지막 실패 사유
data[].manual_request_at	string(date-time)	수동 실행 요청 시각
data[].manual_claimed_at	string(date-time)	수집 노드가 요청을 집어 간 시각
data[].manual_pending	boolean	수동 실행 요청이 아직 집혀 가지 않았다 요청을 넣고 다음 발화를 기다리는 중인지 운영자가 이 값으로 안다
data[].updated_at	string(date-time)	마지막 갱신 시각
data[].steps	array<object>	단계별 현황
data[].steps[].step	string	단계 이름
data[].steps[].status	string	PENDING · RUNNING · SUCCEEDED · FAILED · SKIPPED. 마지막 것은 실패가 아니라 "아직" 이다 — deps.dev 가 그 주 스냅샷을 아직 올리지 않은 경우이고, detail.reason 이 source_not_ready 다
data[].steps[].attempt_count	integer	시도 횟수

필드	타입	설명
<code>data[].steps[].started_at</code>	<code>string(date-time)</code>	시작 시각
<code>data[].steps[].finished_at</code>	<code>string(date-time)</code>	종료 시각
<code>data[].steps[].error_message</code>	<code>string</code>	실패한 단계의 로그 꼬리 4 KB. 전문은 수집 노드의 단계 로그에 있다
<code>data[].steps[].detail</code>	<code>map<string, object></code>	단계별 부가 정보 (예: <code>reason</code>)

오류 V002 · S001

7.2 회차 상세

GET `/api/v1/ops/weekly/runs/{weekOf}`

`weekOf` 는 그 주 월요일(deps.dev 스냅샷 날짜)이다. 다른 요일은 V004 로 거부한다.

Request

이름	위치	타입	필수	설명
<code>weekOf</code>	<code>path</code>	<code>string(date)</code>	Y	회차 주의 월요일 <code>YYYY-MM-DD</code> (deps.dev 스냅샷 날짜). 월요일이 아니면 V004

Response

200 `application/json` — `data: WeeklyRunResponse`

필드	타입	설명
<code>data.week_of</code>	<code>string(date)</code>	회차 주의 월요일 (deps.dev 스냅샷 날짜)
<code>data.status</code>	<code>string</code>	PENDING · RUNNING · SUCCEEDED · FAILED · BLOCKED, 그리고 MISSING — 그 주에 상태 객체가 아예 없다는 뜻이다 타이머가 한 번도 돌지 않았거나 멈춰 있었다
<code>data.coverage</code>	<code>object</code>	이 회차가 덮는 데이터 범위
<code>data.coverage.depsdev_snapshot</code>	<code>string(date)</code>	deps.dev 스냅샷 날짜
<code>data.coverage.downloads_through</code>	<code>string(date)</code>	다운로드 수집 마지막 날짜
<code>data.coverage.window_start</code>	<code>string(date)</code>	수집 구간 시작
<code>data.coverage.window_end</code>	<code>string(date)</code>	수집 구간 끝
<code>data.started_at</code>	<code>string(date-time)</code>	시작 시각
<code>data.finished_at</code>	<code>string(date-time)</code>	종료 시각
<code>data.consecutive_failures</code>	<code>integer</code>	연속 실패 횟수. 한도를 넘으면 BLOCKED
<code>data.last_error</code>	<code>string</code>	마지막 실패 사유
<code>data.manual_request_at</code>	<code>string(date-time)</code>	수동 실행 요청 시각
<code>data.manual_claimed_at</code>	<code>string(date-time)</code>	수집 노드가 요청을 집어 간 시각
<code>data.manual_pending</code>	<code>boolean</code>	수동 실행 요청이 아직 집혀 가지 않았다 요청을 넣고 다음 발화를 기다리는 중인지 운영자가 이 값으로 안다
<code>data.updated_at</code>	<code>string(date-time)</code>	마지막 갱신 시각
<code>data.steps</code>	<code>array<object></code>	단계별 현황
<code>data.steps[].step</code>	<code>string</code>	단계 이름

필드	타입	설명
<code>data.steps[].status</code>	<code>string</code>	PENDING · RUNNING · SUCCEEDED · FAILED · SKIPPED. 마지막 것은 실패가 아니라 "아직" 이다 — <code>deps.dev</code> 가 그 주 스냅샷을 아직 올리지 않은 경우이고, <code>detail.reason</code> 이 <code>source_not_ready</code> 다
<code>data.steps[].attempt_count</code>	<code>integer</code>	시도 횟수
<code>data.steps[].started_at</code>	<code>string(date-time)</code>	시작 시각
<code>data.steps[].finished_at</code>	<code>string(date-time)</code>	종료 시각
<code>data.steps[].error_message</code>	<code>string</code>	실패한 단계의 로그 꼬리 4 KB. 전문은 수집 노드의 단계 로그에 있다
<code>data.steps[].detail</code>	<code>map<string, object></code>	단계별 부가 정보 (예: <code>reason</code>)

오류 V004 · S001

7.3 수동 실행 요청

POST `/api/v1/ops/weekly/runs/{weekOf}/manual-request`

요청 객체만 남긴다. 실제 실행은 `data` 노드 타이머가 다음 발화(최대 10분)에 집어 간다. 연속 실패로 BLOCKED 된 회차를 푸는 유일한 방법이다. 맥등 — 이미 대기 중이어도 200 이다. 아직 오지 않은 주와 다시 받을 수 있는 한계(18개월)를 넘긴 주는 V002 로 거부한다.

Request

이름	위치	타입	필수	설명
<code>weekOf</code>	<code>path</code>	<code>string(date)</code>	Y	회차 주의 월요일 YYYY-MM-DD (<code>deps.dev</code> 스냅샷 날짜). 월요일이 아니면 V004

Response

200 `application/json` — `data: WeeklyRunResponse`

필드	타입	설명
<code>data.week_of</code>	<code>string(date)</code>	회차 주의 월요일 (<code>deps.dev</code> 스냅샷 날짜)
<code>data.status</code>	<code>string</code>	PENDING · RUNNING · SUCCEEDED · FAILED · BLOCKED, 그리고 MISSING — 그 주에 상태 객체가 아예 없다는 뜻이다 타이머가 한 번도 돌지 않았거나 멈춰 있었다
<code>data.coverage</code>	<code>object</code>	이 회차가 덮는 데이터 범위
<code>data.coverage.depsdev_snapshot</code>	<code>string(date)</code>	<code>deps.dev</code> 스냅샷 날짜
<code>data.coverage.downloads_through</code>	<code>string(date)</code>	다운로드 수집 마지막 날짜
<code>data.coverage.window_start</code>	<code>string(date)</code>	수집 구간 시작
<code>data.coverage.window_end</code>	<code>string(date)</code>	수집 구간 끝
<code>data.started_at</code>	<code>string(date-time)</code>	시작 시각
<code>data.finished_at</code>	<code>string(date-time)</code>	종료 시각
<code>data.consecutive_failures</code>	<code>integer</code>	연속 실패 횟수. 한도를 넘으면 BLOCKED
<code>data.last_error</code>	<code>string</code>	마지막 실패 사유
<code>data.manual_request_at</code>	<code>string(date-time)</code>	수동 실행 요청 시각
<code>data.manual_claimed_at</code>	<code>string(date-time)</code>	수집 노드가 요청을 집어 간 시각

필드	타입	설명
<code>data.manual_pending</code>	<code>boolean</code>	수동 실행 요청이 아직 집혀 가지 않았다 요청을 넣고 다음 발화를 기다리는 중인지 운영자가 이 값으로 안다
<code>data.updated_at</code>	<code>string(date-time)</code>	마지막 갱신 시각
<code>data.steps</code>	<code>array<object></code>	단계별 현황
<code>data.steps[].step</code>	<code>string</code>	단계 이름
<code>data.steps[].status</code>	<code>string</code>	<code>PENDING</code> · <code>RUNNING</code> · <code>SUCCEEDED</code> · <code>FAILED</code> · <code>SKIPPED</code> . 마지막 것은 실패가 아니라 "아직" 이다 — <code>deps.dev</code> 가 그 주 스냅샷을 아직 올리지 않은 경우이고, <code>detail.reason</code> 이 <code>source_not_ready</code> 다
<code>data.steps[].attempt_count</code>	<code>integer</code>	시도 횟수
<code>data.steps[].started_at</code>	<code>string(date-time)</code>	시작 시각
<code>data.steps[].finished_at</code>	<code>string(date-time)</code>	종료 시각
<code>data.steps[].error_message</code>	<code>string</code>	실패한 단계의 로그 꼬리 4 KB. 전문은 수집 노드의 단계 로그에 있다
<code>data.steps[].detail</code>	<code>map<string, object></code>	단계별 부가 정보 (예: <code>reason</code>)

오류 `V002` (아직 오지 않은 주:18개월 초과) · `V004` · `S001`

8. 내부 서비스 — RAG 비교 (rag-api)

외부에 노출하지 않는 내부 API. 같은 compose 네트워크의 백엔드만 `http://rag-api:8000` 으로 호출한다. 백엔드 기능 비교(`POST /api/packages/feature-comparison`)가 이 API 를 비동기로 부르고 결과를 `result` 에 싣는다.

8.1 README 근거 기반 비교 생성

POST `/compare`

패키지·버전별 README 인계 파일에서 근거를 추려 LLM 한 번 호출로 공통점과 패키지별 차이를 만들고, 생성물을 근거와 대조해 검증한다.

Request

Body `application/json` (FastAPI 스키마 `CompareRequest`)

필드	타입	설명
<code>packages</code>	<code>array<object></code>	비교 대상 (필수)
<code>packages[].package</code>	<code>string</code>	패키지명 (필수)
<code>packages[].version</code>	<code>string</code>	정확한 버전 (필수)

Response

200 `application/json` — 래퍼 없음, 키는 camelCase

필드	타입	설명
<code>dataStatus</code>	<code>string</code>	비교 가능 여부 상태
<code>packages[]</code>	<code>array<object></code>	비교한 <code>package·version</code> (요청 순서)
<code>common</code>	<code>string</code>	공통점 서술 한 덩어리
<code>differences[]</code>	<code>array<object></code>	패키지별 차이 문단
<code>differences[].package · version · body</code>	<code>string</code>	대상 패키지·버전과 차이 본문
<code>differences[].marks[]</code>	<code>array<object></code>	강조 구간 — 핵심 문장 1개·핵심어 최대 3개. <code>start·end</code> (UTF-16, [start,end)), <code>kind</code>
<code>sources[]</code>	<code>array<object></code>	패키지별 README 인계 파일 상태 — <code>package·version·status·readmeBytes·proseChars</code> (못 읽은 값은 null)

HTTP	본문	조건
404	<code>{"detail": {"code": "DOC_NOT_FOUND", "package", "version"}}</code>	그 버전의 README 인계 파일이 아직 없음
502	<code>{"detail": {"violations": [...]}}</code>	생성물이 근거와 어긋나 검증에서 막음
422	<code>{"detail": [ValidationError]}</code>	요청 본문 형식 오류 (FastAPI 기본)